

Programmation Web Frontend

Session 2 - Version 1.0

L2/X64/06/PWF/187628/26-27

1. Les bases du Web dynamique



1. Les bases du Web dynamique

- ✓ Les langages : niveaux, domaines, paradigmes
- ✓ Qu'est-ce que JavaScript ?
- ✓ Pourquoi apprendre le JavaScript ?
- ✓ Installer son environnement
- ✓ La console
- ✓ Les expressions

Les variables et leurs types

Les opérateurs, les conditions

Les listes

Les boucles

Les primitives

L'objet Math

Les fonctions, les callbacks

Les portées des variables (scope)

Les objets

Les labos : trois niveaux

Même notion, trois profondeurs. Tout le monde fait le niveau 1, puis chacun avance à son rythme.

Niveau 1 · Simple

L'essentiel

- Appliquer la notion telle que vue
- Objectif : tout le monde le termine

Niveau 2 · Modéré

Consolider

- Un mini-scénario concret
- Combiner avec les notions précédentes

Niveau 3 · Avancé

Le défi

- Prédire, déboguer, contourner
- Un bonus pour creuser la documentation

Règle du jeu : uniquement ce qu'on a déjà vu. Pas de **if**, pas de boucle, pas de fonction.

Les variables et leurs types

Qu'est-ce qu'une variable ?

- Jusqu'ici, une valeur était « à usage unique » : affichée, puis perdue
- Une variable = **une boîte étiquetée** : un nom dessus, une valeur dedans

```
const name = "John"
```

name

"John"

```
const age = 40
```

age

40

```
const total = 9.99
```

total

9.99

```
const isAdmin = true
```

isAdmin

true

```
console.log(name, age, total, isAdmin) // John 40 9.99 true
```


Pourquoi les variables ?

Sans variable : trois copies de "John"

```
console.log("Bienvenue, John !")  
console.log("John, 3 nouveaux messages.")  
console.log("Au revoir, John.")
```

Avec une variable : un seul endroit

```
const name = "John"  
  
console.log("Bienvenue, " + name + " !")  
console.log(name + ", 3 nouveaux messages.")  
console.log("Au revoir, " + name + ".")
```

Changez "**John**" en "**Sara**" en haut : **les trois lignes suivent**

let et const

let : la valeur **peut** changer

```
let score = 0
score = 10      // pas de second let
console.log(score) // 10
```

const : la valeur **ne change plus**

```
const name = "Sam"
name = "Alex"
// TypeError: Assignment to constant variable.
```

Anatomie d'une déclaration

let

le mot-clé

« je crée une variable »

score

le nom

l'étiquette de la boîte

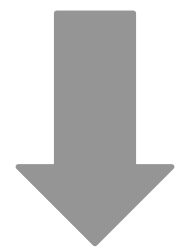
= 0

la valeur initiale

ce qu'on dépose dedans

let ou const ?

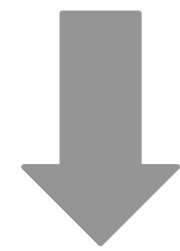
« Est-ce que la valeur va changer ? »



Non

const

le choix par défaut



Oui

let

seulement s'il faut réassigner

Un lecteur qui voit **const** sait que la valeur ne bougera pas : une chose de moins à surveiller.

Et **var** ?

- Mot-clé historique, antérieur à **let** et **const**
- Des pièges pour les débutants
- On y reviendra avec la portée (scope)

D'ici là : n'écrivez pas var

Nommer les variables

Les règles : imposées par JavaScript

- Commence par une lettre, `_` ou `$`
- Ni espace, ni tiret
- Pas un mot réservé : `let`, `if`, `function`...
- Sensible à la casse : `score` ≠ `Score`

Les conventions : imposées par l'équipe

- **camelCase** : `firstName`, `messageCount`
- Booléens : `isLoggedIn`, `hasErrors`, `canEdit`
- Descriptif : `userAge` bat `a` à tous les coups
- Code en anglais, comme dans les équipes

✓ `firstName`

✓ `_count`

✓ `$price`

✓ `total2`

✗ `2fast`

✗ `first-name`

✗ `my name`

✗ `const`

Labo 1 : let et const

Niveau 1 · Simple

Le stock de la boutique

- Un nom de produit en `const`, un stock en `let`
- Affichez les deux
- Deux ventes : réassignez le stock à chaque fois
- Affichez à nouveau

Niveau 2 · Modéré

Le compteur de likes

- Un projet de portfolio : titre et auteur en `const`
- `likes` et `views` en `let`, à 0
- Trois interactions : `likes = likes + 1`
- Affichez les compteurs après chaque interaction
- Tentez de renommer le projet : lisez l'erreur, puis commentez la ligne

Niveau 3 · Avancé

L'inversion de palette

- `primaryColor = "#4A72B8"` et `secondaryColor = "#F2994A"`
- Inversez les deux couleurs **sans retaper** les valeurs
- Affichez avant et après
- Justifiez chaque `let` / `const` en commentaire
- Bonus : que donne `const x` sans valeur ?

Labo 1 · niveau 1

Niveau 1 · Simple

Le stock de la boutique

- Un nom de produit en `const`, un stock en `let`
- Affichez les deux
- Deux ventes : réassignez le stock à chaque fois
- Affichez à nouveau

Hint

Réassigner = réutiliser le nom,
sans `let` : `stock = stock - 1`

Labo 1 · niveau 2

Niveau 2 · Modéré

Le compteur de likes

- Un projet de portfolio : titre et auteur en `const`
- `likes` et `views` en `let`, à 0
- Trois interactions : `likes = likes + 1`
- Affichez les compteurs après chaque interaction
- Tentez de renommer le projet : lisez l'erreur, puis commentez la ligne

Hint

Lisez le message d'erreur en entier : il nomme le problème **et** la ligne

Labo 1 · niveau 3

Niveau 3 · Avancé

L'inversion de palette

- `primaryColor = "#4A72B8"` et `secondaryColor = "#F2994A"`
- Inversez les deux couleurs **sans retaper** les valeurs
- Affichez avant et après
- Justifiez chaque `let` / `const` en commentaire
- Bonus : que donne `const x` sans valeur ?

Hint

Si vous collez la couleur B sur A, où est passée A ? Il vous faut un troisième emplacement.

Solution labo 1 · niveau 1

Niveau 1 · Simple

```
const productName = "Carnet de croquis A5"  
let stock = 10  
console.log(productName, stock)  
  
stock = stock - 1  
stock = stock - 1  
console.log(productName, stock)
```

```
Carnet de croquis A5 10  
Carnet de croquis A5 8
```

SORTIE

Solution labo 1 · niveau 2

Niveau 2 · Modéré

```
const projectTitle = "Refonte app mobile"
const author = "Sara"
let likes = 0
let views = 0

views = views + 1
console.log(projectTitle, "-", likes, "likes,", views, "vues")
views = views + 1
likes = likes + 1
console.log(projectTitle, "-", likes, "likes,", views, "vues")
likes = likes + 1
console.log(projectTitle, "-", likes, "likes,", views, "vues")

// projectTitle = "Nouveau titre"
// TypeError: Assignment to constant variable.
```

```
Refonte app mobile - 0 likes, 1 vuesSORTIE
Refonte app mobile - 1 likes, 2 vues
Refonte app mobile - 2 likes, 2 vues
```


Solution labo 1 · niveau 3

Niveau 3 · Avancé

```
let primaryColor = "#4A72B8"  
let secondaryColor = "#F2994A"  
console.log(primaryColor, secondaryColor)  
  
const temp = primaryColor  
primaryColor = secondaryColor  
secondaryColor = temp  
console.log(primaryColor, secondaryColor)
```

```
#4A72B8 #F2994A  
#F2994A #4A72B8
```

SORTIE

À retenir

- Il faut une **troisième variable** : sans `temp`, la première couleur est écrasée
- `temp` ne change jamais : `const`
- `const x` seul : `SyntaxError`, une promesse sans valeur n'a pas de sens

Les types de données

Chaque valeur a un type, et **le type décide de ce que JavaScript peut en faire**

String

du texte, entre guillemets

```
"John"  
'hello'
```

Number

n'importe quel nombre

```
42  
3.14  
-7
```

Boolean

oui ou non, rien d'autre

```
true  
false
```

null

« pas de valeur », volontairement

```
null
```

undefined

« pas encore défini »

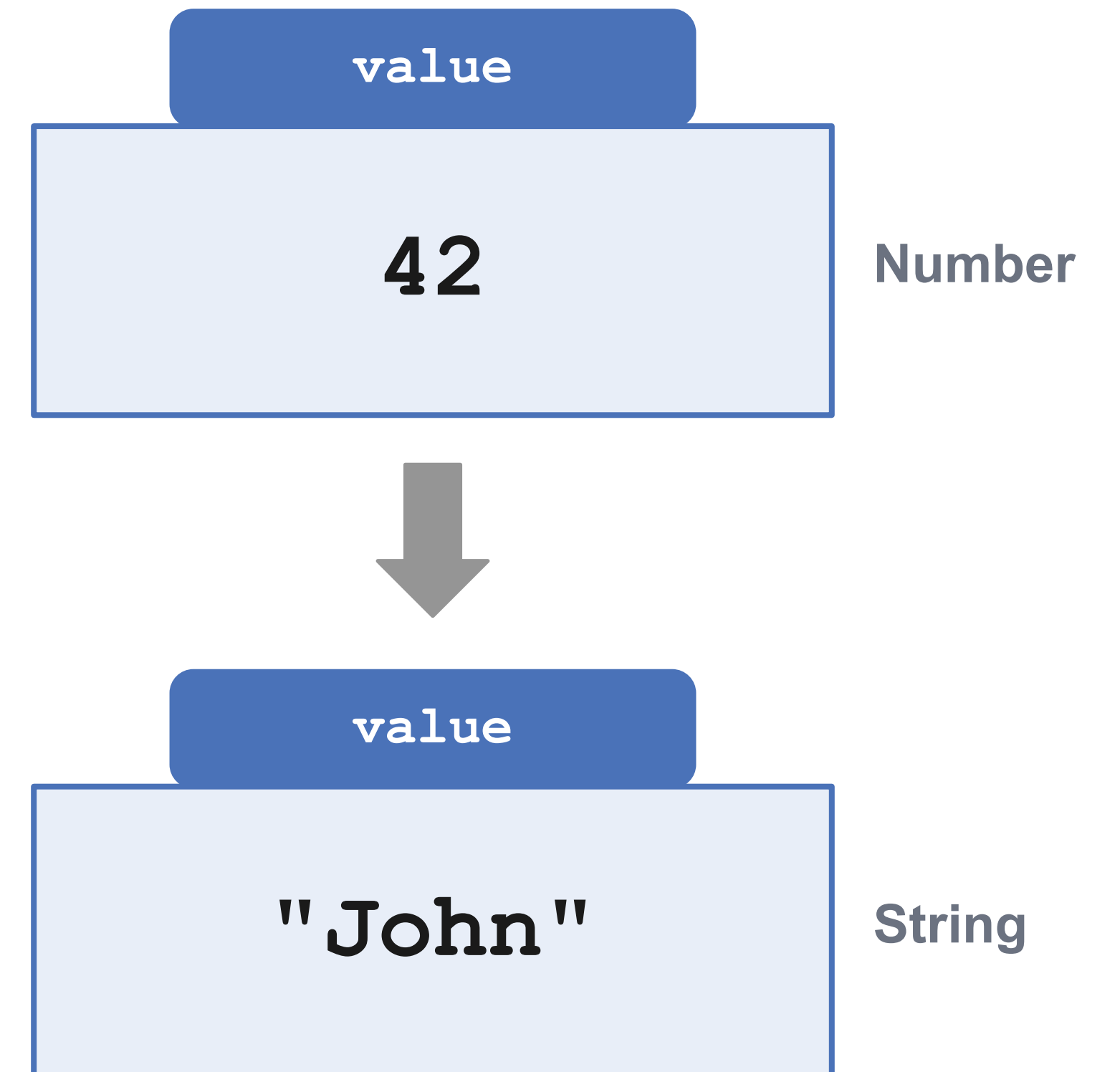
```
undefined
```

Les trois premiers sont courants, les deux derniers sont spéciaux. Objets et tableaux : une autre famille, plus tard.

Typage dynamique

```
let value = 42      // un nombre  
value = "John"     // maintenant un texte  
// JavaScript ne s'en plaint pas
```

- Une variable n'est **pas verrouillée** sur un type
- Pratique pour démarrer, source de bugs plus tard
- D'où l'intérêt de `typeof` et des conversions explicites



Le type appartient à la **valeur**, pas à la variable

Les chaînes de caractères (String)

Trois délimiteurs, **la même chaîne**

```
const single = 'guillemets simples'  
const double = "guillemets doubles"  
const backtick = `accents graves`
```

Un guillemet **dans** le texte ?

```
const broken = "Il a dit "salut""  
// SyntaxError  
const fine = 'Il a dit "salut"'  
const alsoFine = "C'est parti"
```

- Les guillemets marquent le début et la fin : ils ne font **pas partie** de la valeur
- Les doubles sont les plus courants ; les backticks (apostrophe inversée) sont également très utilisés (slide suivant)
- Choisissez le cadrage qui **n'apparaît pas** dans votre texte

Combiner des chaînes

```
const firstName = "John"  
const messages = 3
```

Concaténation avec `+` : la méthode classique

```
console.log("Bonjour, " + firstName +  
  " : " + messages + " messages.")
```

Template literal : la méthode moderne

```
console.log(  
  `Bonjour, ${firstName} : ${messages} messages.`)
```

```
Bonjour, John : 3 messages.
```

SORTIE

`${...}` accepte n'importe quelle expression : ``Total TVAC : ${price * 1.21} €``

Concaténation ou template literal ?

Concaténation +

- Chaque espace, chaque ponctuation est un morceau à part
- Attention au double métier de + : parenthèses pour les calculs
- Courant dans le code existant : `"Score : " + score`

Template literal `\${...}`

- Une seule chaîne, qui se lit comme le résultat
- Convertit les nombres automatiquement
- Conserve les sauts de ligne : parfait pour le multi-ligne

Par défaut : **template literals**

Labo 2 : chaînes de caractères

Niveau 1 · Simple

Le badge

- Trois variables : prénom, studio, rôle
- Un style de guillemets différent pour chacune
- L'une des valeurs contient une apostrophe
- Affichez les trois

Niveau 2 · Modéré

Le message d'accueil

- Un événement, une capacité, un nombre d'inscrits
- Phrase avec les places restantes, par concaténation `+`
- La même phrase en template literal
- Les deux lignes affichées doivent être identiques

Niveau 3 · Avancé

La carte projet du portfolio

- Affichez exactement : `Le client a dit : "C'est parfait !"`
- Une carte projet de 5 lignes en un seul `console.log`
- Au moins un calcul dans un `$ {...}`
- Bonus : que donne `"Total : " + 2 + 3` ? Pourquoi ?

Labo 2 · niveau 1

Niveau 1 · Simple

Le badge

- Trois variables : prénom, studio, rôle
- Un style de guillemets différent pour chacune
- L'une des valeurs contient une apostrophe
- Affichez les trois

Hint

Choisissez le délimiteur qui **n'apparaît pas** dans le texte

Labo 2 · niveau 2

Niveau 2 · Modéré

Le message d'accueil

- Un événement, une capacité, un nombre d'inscrits
- Phrase avec les places restantes, par concaténation +
- La même phrase en template literal
- Les deux lignes affichées doivent être identiques

Hint

Avec +, mettez le calcul entre parenthèses

Labo 2 · niveau 3

Niveau 3 · Avancé

La carte projet du portfolio

- Affichez exactement : `Le client a dit : "C'est parfait !"`
- Une carte projet de 5 lignes en **un seul** `console.log`
- Au moins un calcul dans un `${...}`
- Bonus : que donne `"Total : " + 2 + 3` ? Pourquoi ?

Hint

Un template literal conserve les sauts de ligne tapés entre les backticks

Solution labo 2 · niveau 1

Niveau 1 · Simple

```
const firstName = 'Sara'  
const company = "L'Atelier du Web"  
const role = `Designer UX/UI`  
  
console.log(firstName)  
console.log(company)  
console.log(role)
```

```
Sara  
L'Atelier du Web  
Designer UX/UI
```

SORTIE

Solution labo 2 · niveau 2

Niveau 2 · Modéré

```
const eventName = "Meetup UX Liège"
const capacity = 40
const registered = 28

console.log("Bienvenue au " + eventName + " : " +
  (capacity - registered) + " places libres.")
console.log(
  `Bienvenue au ${eventName} : ${capacity - registered} places libres.`)
```

```
Bienvenue au Meetup UX Liège : 12 places libres.
Bienvenue au Meetup UX Liège : 12 places libres.
```

SORTIE

Solution labo 2 · niveau 3

Niveau 3 · Avancé

```
console.log(`Le client a dit : "C'est parfait !"`)

const title = "Refonte app mobile"
const client = "Friterie du Coin"
const tool = "Figma"
const screens = 12

console.log(`--- Projet ---
Titre   : ${title}
Client  : ${client}
Outil   : ${tool}
Écrans  : ${screens} (dont ${screens - 8} à revoir)`)

console.log("Total : " + 2 + 3)
```

```
Le client a dit : "C'est parfait !"      SORTIE
--- Projet ---
Titre   : Refonte app mobile
Client  : Friterie du Coin
Outil   : Figma
Écrans  : 12 (dont 4 à revoir)
Total   : 23
```

À retenir

- Backticks : les deux autres guillemets deviennent neutres
- Les sauts de ligne du template literal sont conservés
- `"Total : " + 2` donne du texte, puis `+ 3` colle encore : lecture de gauche à droite

Les nombres (Number)

- Entiers, décimaux, négatifs : **un seul type** Number
- Pas de guillemets, sinon c'est du texte
- Les variables se calculent comme des valeurs

42

Number

"42"

String

Ils se ressemblent, JavaScript les traite très différemment

```
const price = 100
const vatRate = 0.21

const vat = price * vatRate
const total = price + vat

console.log(total)    // 121
```

Arrondis, aléatoire, **NaN**, **Infinity** : un chapitre complet plus tard

Labo 3 : nombres

Niveau 1 · Simple

De px à rem

- Une taille de police en px, une base de 16 px
- Calculez la taille en rem : `px / base`
- Affichez les deux valeurs

Niveau 2 · Modéré

La grille responsive

- Un conteneur de 1200 px, 12 colonnes, gap de 24 px
- Calculez le total des gaps, puis la largeur d'une colonne
- Calculez la largeur d'une carte sur 4 colonnes
- Chaque résultat dans sa propre `const`, affiché en template literal

Niveau 3 · Avancé

Le panier de la boutique

- Trois articles à prix décimaux
- Remise de 10 %, puis TVA de 21 %
- Affichez l'économie et le total TVAC
- Observez `0.1 + 0.2`. Surpris ?
- Obtenez 0.3 en calculant en centimes

Labo 3 · niveau 1

Niveau 1 · Simple

De px à rem

- Une taille de police en px, une base de 16 px
- Calculez la taille en rem : px / base
- Affichez les deux valeurs

Hint

24 px sur une base de 16 px = 1.5 rem

Labo 3 · niveau 2

Niveau 2 · Modéré

La grille responsive

- Un conteneur de 1200 px, 12 colonnes, gap de 24 px
- Calculez le total des gaps, puis la largeur d'une colonne
- Calculez la largeur d'une carte sur 4 colonnes
- Chaque résultat dans sa propre `const`, affiché en template literal

Hint

12 colonnes = seulement 11 gouttières entre elles

Labo 3 · niveau 3

Niveau 3 · Avancé

Le panier de la boutique

- Trois articles à prix décimaux
- Remise de 10 %, puis TVA de 21 %
- Affichez l'économie et le total TVAC
- Observez $0.1 + 0.2$. Surpris ?
- Obtenez 0.3 en calculant en centimes

Hint

1 € = 100 centimes : additionnez des entiers, divisez à la fin

Solution labo 3 · niveau 1

Niveau 1 · Simple

```
const fontSizePx = 24
const baseSizePx = 16
const fontSizeRem = fontSizePx / baseSizePx

console.log(fontSizePx)
console.log(fontSizeRem)
```

```
24
1.5
```

SORTIE

Solution labo 3 · niveau 2

Niveau 2 · Modéré

```
const containerWidth = 1200
const columns = 12
const gap = 24

const totalGaps = (columns - 1) * gap
const columnWidth = (containerWidth - totalGaps) / columns
const cardWidth = columnWidth * 4 + gap * 3

console.log(`Gaps : ${totalGaps}px`)
console.log(`Une colonne : ${columnWidth}px`)
console.log(`Carte sur 4 colonnes : ${cardWidth}px`)
```

```
Gaps : 264px
Une colonne : 78px
Carte sur 4 colonnes : 384px
```

SORTIE

Solution labo 3 · niveau 3

Niveau 3 · Avancé

```
const poster = 24.9
const stickers = 4.95
const toteBag = 12.5
const discountRate = 0.1
const vatRate = 0.21

const subtotal = poster + stickers + toteBag
const discount = subtotal * discountRate
const total = (subtotal - discount) * (1 + vatRate)
console.log(`Économie : ${discount} €`)
console.log(`Total TVAC : ${total} €`)

console.log(0.1 + 0.2)
console.log((10 + 20) / 100)
```

```
Économie : 4.234999999999999 €      SORTIE
Total TVAC : 46.11914999999999 €
0.30000000000000004
0.3
```

À retenir

- Un ordinateur compte en binaire, et 0.1 n'a pas d'écriture exacte en binaire, comme 1/3 en décimal. D'où de minuscules erreurs d'arrondi.
- Pour l'argent : calculer **en centimes** (des entiers)
- Arrondir à l'affichage : chapitre Math

Les booléens (Boolean)

- **true** ou **false**, c'est tout
- C'est ainsi qu'un programme **décide** : « connecté ? », « panier vide ? »
- Indispensables pour les **if** (prochain chapitre)

```
const isLoggedIn = true
const isAdmin = false
const hasErrors = false
const canEdit = true
```

Convention : un nom qui se lit comme une question

is

isActive

has

hasErrors

can

canEdit

null et undefined

null

« J'ai vérifié, et il n'y a rien. »

```
const middleName = null
```

C'est **vous** qui l'avez mis

undefined

« Personne ne m'a donné de valeur. »

```
let username  
console.log(username) // undefined
```

C'est **JavaScript** qui l'a mis

Les deux signifient « rien »... mais pas pour la même raison

typeof

Un opérateur qui renvoie le type d'une valeur, **sous forme de chaîne**

`typeof "John"` → `"string"`

`typeof 42` → `"number"`

`typeof true` → `"boolean"`

`typeof undefined` → `"undefined"`

`typeof null` → `"object"`

Un outil de débogage

- Une valeur se comporte bizarrement ?
- `console.log(typeof x)`

`typeof null` → `"object"` : bug historique, impossible à corriger sans casser le web.

Labo 4 : booléens, null, undefined, typeof

Niveau 1 · Simple

Le composant bouton

- `isDisabled` : un booléen
- `icon` : volontairement vide
- `label` : déclarée, sans valeur
- Affichez la valeur **et** le `typeof` de chacune

Niveau 2 · Modéré

La variable caméléon

- Une seule variable `let value`
- Donnez-lui tour à tour : rien, `5`, `"5"`, `true`, `null`
- Après chaque étape, affichez `value` et `typeof value`
- Quelle ligne vous surprend ?

Niveau 3 · Avancé

Prédire avant d'exécuter

- Notez votre prédiction en commentaire, puis vérifiez :
- `typeof typeof 42`
- `typeof `${42}``
- `typeof 42 + " ans"`
- `typeof (42 + " ans")`
- `typeof notDeclared`

Labo 4 · niveau 1

Niveau 1 · Simple

Le composant bouton

- `isDisabled` : un booléen
- `icon` : volontairement vide
- `label` : déclarée, sans valeur
- Affichez la valeur **et** le `typeof` de chacune

Hint

`console.log` accepte plusieurs valeurs séparées par des virgules

Labo 4 · niveau 2

Niveau 2 · Modéré

La variable “multi-forme”

- Une seule variable `let value`
- Donnez-lui tour à tour : rien, `5`, `"5"`, `true`, `null`
- Après chaque étape, affichez `value` et `typeof value`
- Quelle ligne vous surprend ?

Hint

Une seule déclaration `let`,
ensuite uniquement des
réassignations

Labo 4 · niveau 3

Niveau 3 · Avancé

Prédire avant d'exécuter

- Notez votre prédiction en commentaire, puis vérifiez :
- `typeof typeof 42`
- `typeof `${42}``
- `typeof 42 + " ans"`
- `typeof (42 + " ans")`
- `typeof notDeclared`

Hint

`typeof` s'applique à ce qui le suit immédiatement. Et il renvoie toujours... quoi ?

Solution labo 4 · niveau 1

Niveau 1 · Simple

```
const isDisabled = false
const icon = null
let label

console.log(isDisabled, typeof isDisabled)
console.log(icon, typeof icon)
console.log(label, typeof label)
```

```
false boolean
null object
undefined undefined
```

SORTIE

Solution labo 4 · niveau 2

Niveau 2 · Modéré

```
let value
console.log(`${value} -> ${typeof value}`)
value = 5
console.log(`${value} -> ${typeof value}`)
value = "5"
console.log(`${value} -> ${typeof value}`)
value = true
console.log(`${value} -> ${typeof value}`)
value = null
console.log(`${value} -> ${typeof value}`)
```

```
undefined -> undefined
5 -> number
5 -> string
true -> boolean
null -> object
```

SORTIE

Solution labo 4 · niveau 3

Niveau 3 · Avancé

Expression	Résultat	Pourquoi ?
<code>typeof typeof 42</code>	<code>"string"</code>	<code>typeof</code> renvoie toujours une chaîne : <code>typeof "number"</code>
<code>typeof `\${42}`</code>	<code>"string"</code>	Un template literal produit toujours du texte
<code>typeof 42 + " ans"</code>	<code>"number ans"</code>	<code>typeof</code> s'applique d'abord à 42, puis + concatène
<code>typeof (42 + " ans")</code>	<code>"string"</code>	Les parenthèses d'abord : "42 ans" est un texte
<code>typeof notDeclared</code>	<code>"undefined"</code>	Seul cas où une variable jamais déclarée ne provoque pas d'erreur

`typeof` est prioritaire sur `+` : en cas de doute, **des parenthèses**

Conversion de types : explicite

Un nombre saisi dans un formulaire est **du texte**. Trois fonctions pour convertir volontairement :

String(42)



"42"

vers du texte

Number("42")



42

vers un nombre

Boolean(1)



true

vers un booléen

Number("hello") → NaN

Not a Number : la chaîne n'est pas un nombre valide

Boolean(...) → presque tout devient **true**

Sauf 0, "", **null**, **undefined**, **NaN** (chapitre suivant)

Conversion implicite (coercition)

JavaScript convertit aussi **dans votre dos**. C'est là que naissent les bugs.

"5" + 1 → "51"

+ a deux métiers : dès qu'il voit du texte, il **colle**

"5" - 1 → 4

- ne sait que **calculer** : il convertit "5" en 5

Le danger : le programme ne plante pas, il produit **un résultat faux, en silence**

En cas de doute, **convertissez explicitement** : `Number()` avant un calcul, template literal pour du texte

Labo 5 : conversion de types

Niveau 1 · Simple

Le champ « largeur »

- `widthInput = "320"` (saisi dans un champ !) et `padding = 16`
- Affichez `widthInput + padding` : que se passe-t-il ?
- Corrigez avec `Number()`
- Prouvez le changement avec `typeof` avant / après

Niveau 2 · Modéré

La réservation au resto

- `guestsInput = "4"` vient du formulaire, menu à 32 €
- Calculez le total, construisez `"128 €"` avec `String()`
- Testez `Boolean()` sur `""`, `"0"`, `0` et `"false"`
- Lequel est un piège pour une case « newsletter » ?

Niveau 3 · Avancé

Prédire avant d'exécuter

- Prédiction en commentaire, puis vérifiez :
- `1 + 2 + "3"` et `"1" + 2 + 3`
- `"5" * "2"`
- `Number("")` et `Number("12px")`
- `Number(null)` et `Number(undefined)`
- `Boolean("false")`

Labo 5 · niveau 1

Niveau 1 · Simple

Le champ « largeur »

- `widthInput = "320"` (saisi dans un champ !) et `padding = 16`
- Affichez `widthInput + padding` : que se passe-t-il ?
- Corrigez avec `Number()`
- Prouvez le changement avec `typeof` avant / après

Hint

+ avec un texte : il colle. Regardez bien le résultat affiché

Labo 5 · niveau 2

Niveau 2 · Modéré

La réservation au resto

- `guestsInput = "4"` vient du formulaire, menu à 32 €
- Calculez le total, construisez `"128 €"` avec `String()`
- Testez `Boolean()` sur `"", "0", 0` et `"false"`
- Lequel est un piège pour une case « newsletter » ?

Hint

Un formulaire HTML envoie toujours du texte : `"true"`, `"false"`, `"`...

Labo 5 · niveau 3

Niveau 3 · Avancé

Prédire avant d'exécuter

- Prédiction en commentaire, puis vérifiez :
- `1 + 2 + "3"` et `"1" + 2 + 3`
- `"5" * "2"`
- `Number("")` et `Number("12px")`
- `Number(null)` et `Number(undefined)`
- `Boolean("false")`

Hint

Lisez de gauche à droite, une opération à la fois

Solution labo 5 · niveau 1

Niveau 1 · Simple

```
const widthInput = "320"  
const padding = 16  
  
console.log(widthInput + padding)  
console.log(typeof widthInput)  
console.log(typeof Number(widthInput))  
console.log(Number(widthInput) + padding)
```

```
32016  
string  
number  
336
```

SORTIE

Solution labo 5 · niveau 2

Niveau 2 · Modéré

```
const guestsInput = "4"
const menuPrice = 32

const total = Number(guestsInput) * menuPrice
const label = String(total) + " €"
console.log(label, typeof label)

console.log(Boolean(""))
console.log(Boolean("0"))
console.log(Boolean(0))
console.log(Boolean("false"))
```

```
128 € string
false
true
false
true
```

SORTIE

Solution labo 5 · niveau 3

Niveau 3 · Avancé

Expression	Résultat	Pourquoi ?
<code>1 + 2 + "3"</code>	<code>"33"</code>	De gauche à droite : $1 + 2 = 3$, puis $3 + "3"$ colle
<code>"1" + 2 + 3</code>	<code>"123"</code>	Dès le premier <code>+</code> , on est en mode texte
<code>"5" * "2"</code>	<code>10</code>	<code>*</code> ne sait que calculer : les deux textes sont convertis
<code>Number("")</code>	<code>0</code>	Chaîne vide = 0 : un champ vide passe pour un zéro !
<code>Number("12px")</code>	<code>NaN</code>	Un seul caractère invalide suffit
<code>Number(null)</code>	<code>0</code>	<code>null</code> = « rien » devient 0
<code>Number(undefined)</code>	<code>NaN</code>	<code>undefined</code> = « inconnu » devient NaN
<code>Boolean("false")</code>	<code>true</code>	Toute chaîne non vide est vraie, même "false"

La règle : **convertissez explicitement !**

Projet : le ticket de caisse

- Premier programme complet
- Uniquement ce que vous connaissez : pas de `if`, pas de boucle, pas de liste
- Toutes les valeurs affichées viennent de **variables**
- Le commerce et le contenu sont libres : friterie, librairie, cinéma...

```
=====
Friterie du Coin
=====
Client : Sara
2 x Grande frite + sauce
Code promo : null
Carte fidélité : true
Retrait : undefined
Total : 9.8 €
=====
```

SORTIE

Exemple de résultat attendu (niveau 2)

Labo 6 : le ticket de caisse

Niveau 1 · Simple

Le ticket minimal

- Au moins une `const` et une `let`
- Un template literal avec `${...}`
- Une concaténation avec `+`
- Un total **calculé**, pas tapé à la main
- Au moins 3 `console.log`

Niveau 2 · Modéré

Le ticket complet

- Les **cinq types** primitifs sont présents
- La quantité arrive d'un formulaire : `"2"`, à convertir
- Un supplément s'ajoute au total : `let` prend son sens
- Un cadre `=====` autour du ticket

Niveau 3 · Avancé

Le ticket pro

- Tout le ticket en **un seul** `console.log`
- Une ligne « debug » : le `typeof` de trois valeurs
- Changez une valeur en haut : tout le ticket suit
- Bonus : générez le cadre avec `repeat()` (à chercher sur MDN)

Labo 6 · niveau 1

Niveau 1 · Simple

Le ticket minimal

- Au moins une `const` et une `let`
- Un template literal avec `${...}`
- Une concaténation avec `+`
- Un total **calculé**, pas tapé à la main
- Au moins 3 `console.log`

Hint

Déclarez toutes vos variables en haut, affichez ensuite

Labo 6 · niveau 2

Niveau 2 · Modéré

Le ticket complet

- Les **cinq types** primitifs sont présents
- La quantité arrive d'un formulaire : `"2"`, à convertir
- Un supplément s'ajoute au total : `let` prend son sens
- Un cadre `=====` autour du ticket

Hint

`undefined` : une information pas encore connue, comme l'heure de retrait

Labo 6 · niveau 3

Niveau 3 · Avancé

Le ticket pro

- Tout le ticket en **un seul** `console.log`
- Une ligne « debug » : le `typeof` de trois valeurs
- Changez une valeur en haut : tout le ticket suit
- Bonus : générez le cadre avec `repeat()` (à chercher sur MDN)

Hint

MDN : cherchez
« `String.prototype.repeat` », lisez
l'exemple, testez-le

Solution labo 6 · niveau 1

Niveau 1 · Simple

```
const shopName = "Friterie du Coin"
const customer = "Sara"
const unitPrice = 4.5
let quantity = 2

console.log("=====")
console.log("  " + shopName)
console.log("=====")
console.log(`Client : ${customer}`)
console.log(`${quantity} x Grande frite : ${unitPrice * quantity} €`)
console.log("=====")
```

```
===== SORTIE
  Friterie du Coin
=====
Client : Sara
2 x Grande frite : 9 €
=====
```


Solution labo 6 · niveau 2

Niveau 2 · Modéré

```
const shopName = "Friterie du Coin"
const customer = "Sara"
const unitPrice = 4.5
const quantityInput = "2"
const saucePrice = 0.8
const promoCode = null
const isMember = true
let pickupTime

let total = unitPrice * Number(quantityInput)
total = total + saucePrice

console.log("=====")
console.log("  " + shopName)
console.log("=====")
console.log(`Client : ${customer}`)
console.log(`${quantityInput} x Grande frite + sauce`)
console.log(`Code promo : ${promoCode}`)
console.log(`Carte fidélité : ${isMember}`)
console.log(`Retrait : ${pickupTime}`)
console.log(`Total : ${total} €`)
console.log("=====")
```

```
===== SORTIE
  Friterie du Coin
=====
Client : Sara
2 x Grande frite + sauce
Code promo : null
Carte fidélité : true
Retrait : undefined
Total : 9.8 €
=====
```

Solution labo 6 · niveau 3

Niveau 3 · Avancé

```
const shopName = "Friterie du Coin"
const customer = "Sara"
const unitPrice = 4.5
const quantityInput = "2"
const promoCode = null
const isMember = true

const quantity = Number(quantityInput)
const total = unitPrice * quantity
const line = "=".repeat(30)

console.log(`${line}
  ${shopName}
${line}
Client : ${customer}
${quantity} x Grande frite : ${total} €
Code promo : ${promoCode}
Carte fidélité : ${isMember}
${line}
debug : ${typeof quantityInput} / ${typeof quantity} / ${typeof promoCode}`)
```

```
===== SORTIE
  Friterie du Coin
=====
Client : Sara
2 x Grande frite : 9 €
Code promo : null
Carte fidélité : true
=====
debug : string / number / object
```


En résumé

Variable

Un espace mémoire contenant une valeur

const / let

`const` par défaut, `let` si ça change, jamais `var`

Nommage

camelCase, descriptif, en anglais

Cinq types

String, Number, Boolean, null, undefined

Typage dynamique

Le type appartient à la valeur, pas à la variable

Texte

Template literals par défaut, `+` en connaissance de cause

typeof

Pour vérifier ce que JavaScript pense d'une valeur

Conversion

Explicite : `Number()`, `String()`, `Boolean()`